Credit Card Fraud Detection



Elena Mazzilli

- New gegeration data models and DBMSs | Università degli studi di Milano

CCFD

Graph-Based Transaction Analysis

IMPORT DATA

GRAPH ANALYSIS



Project showcase: **1.28M Transactions | 3 DATASETS sizes**

Creation of a NoSQL Database for Credit Card Fraud Detection

CCFD

From isolated payments to connected patterns

A graph-oriented approach to exploring financial transaction behavior.

THE CHALLENGE

Transactions are not independent.

Potential fraud signals can emerge from relationships across customers, terminals and time.

SHARED TERMINALS	Different customers, overlapping payment locations
UNUSUAL AMOUNT	Transaction values outside typical terminal behavior
MOLTI-HOP LINKS	Indirect relationships between customers

THE APPROACH



After obtained synthetic transaction datasets, the main goal design a NoSQL data model in Neo4j to execute and optimize queries relevant to fraud detection.

CCFD

From isolated payments to connected patterns

A graph-oriented approach to exploring financial transaction behavior.

THE CHALLENGE

Transactions are not independent.

Potential fraud signals can emerge from relationships across customers, terminals and time.

SHARED TERMINALS	Different customers, overlapping payment locations
UNUSUAL AMOUNT	Transaction values outside typical terminal behavior
MOLTI-HOP LINKS	Indirect relationships between customers

THE APPROACH

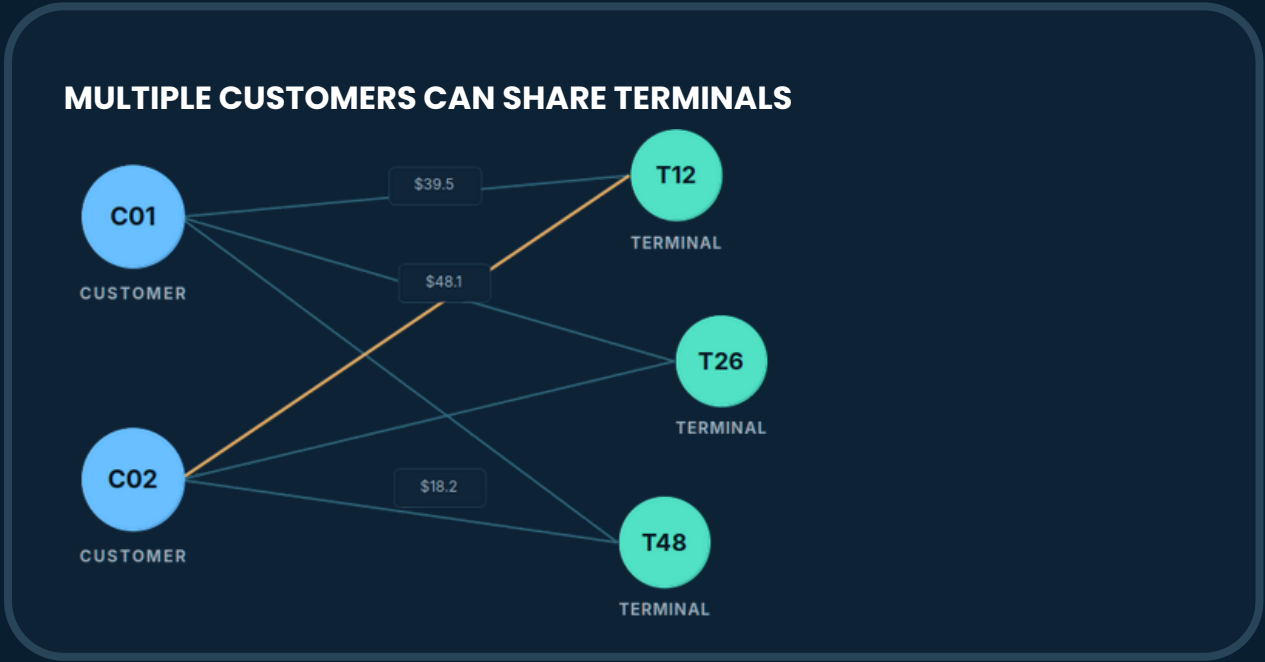


After obtained synthetic transaction datasets, the main goal design a NoSQL data model in Neo4j to execute and optimize queries relevant to fraud detection.

CCFD

A transaction is a relationship

Neo4j links customer profiles and terminals through transactions carrying payment metadata.



GRAPH SCHEMA

SYNTHETIC
ID, location, spending profile

TERMINAL
ID, geographical coordinates

TRANSACTION
Amount, timestamp, fraud label

- generation of customer profiles
- generation of terminal profiles
- association of customers profiles to terminals
- generating of transactions
- generating fraud scenarios

CCFD

ENGINEERING PIPELINE

From generated data to a queryable graph

A repeatable workflow for ingestion, indexing and benchmark datasets.

1 Generate

Customer profiles, terminals and synthetic transactions

2 Prepare

CSV datasets with fraud flags and transaction properties

3 Import

Neo4j indexes, nodes and transaction relationships

BENCHMARK DATASETS

50 MB	**315,861**	
TRANSACTIONS	90 days	
100 MB	**631,830**	
TRANSACTIONS	180 days	
200 MB	**1,280,958**	
TRANSACTIONS	365 days	

Fixed in every dataset: 2,000 customers · 350 terminals

CCFD

Three analytical questions answered with Cypher

Rule-based queries expose behavioral patterns and connections across transaction data.

QUERY A

Shared behavior

Identify customers sharing at least three terminals, with spending amounts differing by less than 10%.

SIMILARITY

QUERY B

Amount anomalies

Compare terminal transactions against a reference computed from the previous month's payment amounts.

DEVIATION

QUERY C

Co-customer paths

Traverse chains of customers linked by shared terminals to find indirect relationships (CC_3).

CONNECTIVITY

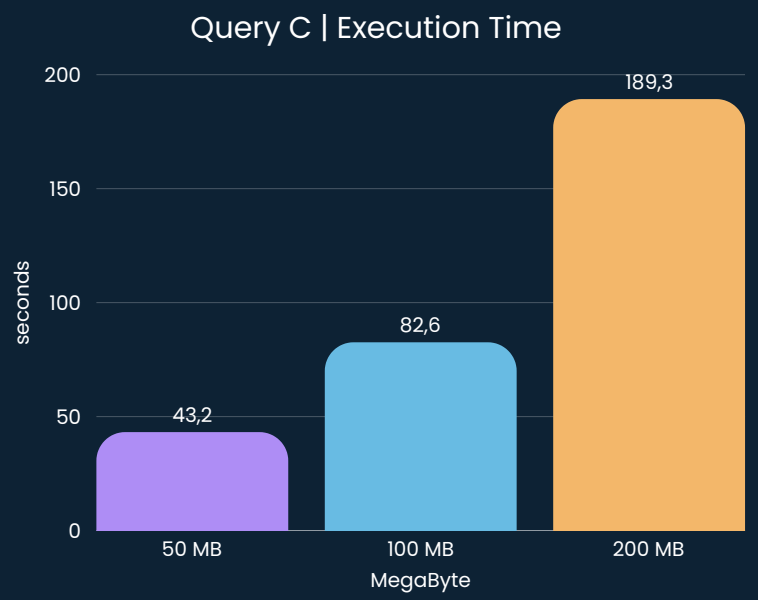
MODEL EXTENSIONS Period of day · Product type · Security rating · "Buying friends" links

CCFD

Relationship depth is costly

Query C: execution time for co-customer relationship analysis, measured on three dataset sizes.

RESULTS



RESULTS

4.4× | longer execution time

for Query C from the smallest to the largest benchmark dataset.

IMPROVEMENT DIRECTIONS

Constrained traversals, precomputed links and batched updates.

CCFD

What the project demonstrates

The value of graph modeling, careful query design and measured engineering decisions.

1 DATA MODELING

Turn tabular transaction records into an expressive relationship-oriented Neo4j schema.

2 QUERY ENGINEERING

Translate analytical requirements into Cypher queries for patterns, anomalies and paths.

3 PERFORMANCE THINKING

Compare execution times at increasing data volumes and reason about query complexity.

TECHNOLOGIES **Python · Neo4j · Cypher · Jupiter Notebook**